

Modul SYNESIS

Satu dokumen, semua konsep

Sandy Fauzi Amrulloh

Agustus 2026

Contents

Modul: satu dokumen, semua konsep	1
Cara memakai dokumen ini	1
Bagian 0. Satu ide yang menjelaskan hampir segalanya	2
Bagian 1. Mesin belajar itu apa sebenarnya	3
Bagian 2. Menghafal lawan memahami	5
Bagian 3. Dari garis lurus ke jaringan saraf	7
Bagian 4. Mengubah dunia jadi angka	10
Bagian 5. Suara	12
Bagian 6. Wajah	15
Bagian 7. Bahasa	16
Bagian 8. Merakit jadi asisten	19
Bagian 9. Kamus fisika ke machine learning	21
Bagian 10. Tes jujur, apakah kamu benar-benar paham	22
Bagian 11. Peta satu halaman	23
Catatan penutup	24

Modul: satu dokumen, semua konsep

Ini bukan silabus dan bukan rencana kerja. Ini penjelasannya.

Silabus.md memberi tahu kamu harus belajar apa dan kapan. Dokumen ini memberi tahu barangnya itu sebenarnya apa, dijelaskan dengan gambaran yang bisa kamu bayangkan, bukan dengan definisi yang bisa kamu hafal.

Cara memakai dokumen ini

Baca satu bagian, lalu tutup layarnya dan jelaskan ulang dengan suara keras ke tembok. Kalau macet di tengah kalimat, itu bukan tanda kamu kurang pintar. Itu tanda kamu tahu namanya tapi belum tahu barangnya, dan kamu baru saja menemukan lokasi persis lubangnya.

Di tiap bagian ada tiga hal yang sengaja saya pasang:

Gambaran konkret yang bisa kamu lihat di kepala. Bukan rumus dulu, benda dulu.

Catatan **di mana analoginya rusak**. Semua analogi bohong sedikit. Kalau kamu tidak tahu di mana bohongnya, kamu akan salah paham pada kasus tepi, dan kasus tepi itulah yang bikin kodemu diam-diam salah.

Blok **tanya diri sendiri** di akhir tiap bagian. Jawab dengan suara keras. Yang tidak bisa kamu jawab, itu materi belajarmu minggu itu.

Satu peringatan sebelum mulai. Dokumen ini akan bikin kamu merasa paham. Perasaan itu menipu. Perasaan paham datang dari membaca, tapi pemahaman datang dari mengetik kode yang error lalu memperbaikinya. Dokumen ini cuma peta. Jalannya tetap harus kamu lewati sendiri.

Bagian o. Satu ide yang menjelaskan hampir segalanya

Sebelum masuk ke mana pun, pahami ini dulu. Kalau cuma satu hal yang kamu bawa dari dokumen ini, ambil yang ini.

Bayangkan sebuah mangkuk. Kamu taruh kelereng di pinggirnya, lalu lepas. Kelereng menggelinding turun, sempat naik lagi ke seberang, bolak-balik, akhirnya berhenti di dasar.

Kamu sudah tahu kenapa. Kelereng selalu bergerak ke arah energi potensial yang lebih rendah.

Sekarang: **seluruh machine learning adalah ini**.

Regresi linear di Modul 0 adalah ini. Melatih MNIST di Modul 1 adalah ini. GPT-4 dilatih dengan cara ini. ChatGPT, Midjourney, model yang mengenali wajahmu di HP, semuanya kelereng yang menggelinding di mangkuk.

Yang berubah cuma dua hal, dan tidak satu pun mengubah idenya:

Mangkuknya bukan di ruang 3D. Ia di ruang berdimensi sebanyak jumlah parameter model. Model kecilmu punya 2 parameter, jadi mangkuknya betulan permukaan 2D yang bisa kamu plot dan lihat. GPT-4 punya ratusan miliar parameter, jadi mangkuknya berdimensi ratusan miliar dan tidak ada manusia yang bisa membayangkannya. Tapi matematikanya identik.

Mangkuknya tidak berbentuk mangkuk. Model linear menghasilkan mangkuk beneran, mulus, satu dasar. Jaringan dalam menghasilkan sesuatu yang lebih mirip pegunungan Himalaya berdimensi jutaan, penuh lembah, punggung, dan dataran datar yang bikin frustrasi. Kelerengnya tetap menggelinding turun. Ia cuma tidak dijamin sampai ke lembah yang paling dalam.

Itu saja. Sisa dokumen ini menjelaskan cara membangun mangkuknya, cara merasakan kemiringannya, dan cara melangkah tanpa terpelanting.

Di mana analoginya rusak. Kelereng punya massa dan momentum, jadi ia melewati dasar lalu berayun balik. Gradient descent polos tidak punya momentum. Ia tidak melewati dasar karena inersia, ia melewatinya karena langkahnya kebesaran, dan itu sebab yang berbeda. Menariknya, optimizer bernama Adam dan SGD-with-momentum justru sengaja menambahkan momentum buatan, dan analogi kelereng jadi lebih akurat di situ ketimbang di gradient descent biasa.

Bagian 1. Mesin belajar itu apa sebenarnya

Model adalah mesin berkenop

Bayangkan kotak dengan satu lubang masuk, satu lubang keluar, dan beberapa kenop di atasnya. Kamu masukkan angka lewat lubang masuk. Keluar angka lain dari lubang keluar. Angka apa yang keluar tergantung posisi kenop.

Itu model. Betulan cuma itu.

Model paling sederhana punya dua kenop, w dan b , dan aturannya $\text{keluar} = w * \text{masuk} + b$. Putar w , garisnya jadi lebih curam. Putar b , garisnya naik turun.

Model pengenalan wajah punya beberapa juta kenop. Qwen3-4B punya empat miliar kenop. Aturannya jauh lebih berbelit. Tapi kalimatnya tetap sama: masuk angka, keluar angka, hasilnya tergantung posisi kenop.

“Melatih model” artinya mencari posisi kenop yang benar. Titik.

Dan sekarang perhatikan sesuatu yang penting. Kenopnya tidak diputar manusia. Kalau diputar manusia, dua kenop pun sudah menyiksa, apalagi empat miliar. Yang kita cari adalah **cara supaya kenopnya memutar dirinya sendiri**.

Loss adalah skor kesalahan

Untuk membiarkan kenop memutar dirinya sendiri, mesin butuh tahu seberapa salah dia sekarang. Satu angka. Makin kecil makin bagus.

Angka itu namanya loss.

Cara membuatnya lurus saja. Kamu punya jawaban yang benar. Kamu punya tebakan mesin. Kurangi, lalu kuadratkan, lalu rata-ratakan seluruh data:

$$\text{MSE} = (1/n) * \text{jumlah dari } (\text{tebakan} - \text{jawaban})^2$$

Kenapa dikuadratkan, bukan dijumlah begitu saja? Dua alasan, dan keduanya beralasan.

Pertama, tanpa dikuadratkan, meleset ke atas 5 dan meleset ke bawah 5 akan saling menghapus jadi nol. Mesinnya akan mengira dirinya sempurna padahal dua-duanya salah.

Kedua, kuadrat menghukum kesalahan besar jauh lebih keras. Meleset 10 kali sebesar 1 memberi skor 10. Meleset sekali sebesar 10 memberi skor 100. Jadi model akan mati-matian menghindari kesalahan besar dan agak santai pada kesalahan kecil. Kadang itu yang kamu mau, kadang tidak, dan di situlah MAE masuk sebagai alternatif.

Ada satu hal yang mengejutkan orang saat pertama kali melihatnya, dan kamu sudah melihatnya sendiri di Sesi 3. **Loss di parameter yang benar-benar asli pun tidak nol.**

Datamu punya derau. Bahkan $w = 3$, $b = 2$ yang betulan melahirkan data itu tidak bisa melewati setiap titik, karena setiap titik sudah digeser acak. Loss yang tersisa itu bukan kegagalan model. Itu deraunya sendiri, dan tidak ada model di alam semesta ini yang bisa menghapusnya.

Angka itu adalah lantai. Model yang loss-nya menembus ke bawah lantai bukan model yang hebat. Itu model yang mulai menghafal deraunya, dan kita akan bahas penyakit itu di Bagian 2.

Gradien adalah kemiringan di bawah kakimu

Kamu berdiri di lereng gunung. Kabut tebal. Jarak pandang nol. Kamu mau turun ke lembah.

Kamu tidak bisa melihat lembah. Kamu tidak bisa melihat apa pun. Tapi ada satu hal yang tetap bisa kamu rasakan: kemiringan tanah tepat di bawah telapak kakimu. Kamu bisa merasakan arah mana yang menurun paling tajam.

Itu gradien. Persis itu.

Secara matematis gradien adalah kumpulan turunan parsial, satu untuk tiap parameter. Untuk model dua kenopmu, gradien adalah dua angka: dL/dw dan dL/db . Angka pertama menjawab “kalau w saya naikkan sedikit, loss-nya naik atau turun, dan seberapa cepat?” Angka kedua menanyakan hal yang sama untuk b .

Gabungan dua angka itu menunjuk ke arah **paling menanjak**. Jadi kamu melangkah ke arah kebalikannya. Itu sebabnya ada tanda minus di rumus pembaruan parameter, dan itu satu-satunya alasan tanda minus itu ada.

Di mana analoginya rusak. Di gunung beneran, kalau kabutnya hilang sebentar kamu bisa melihat ke sekeliling dan langsung tahu lembah yang benar ada di sebelah mana. Di gradient descent, kabutnya tidak pernah hilang. Selamanya. Kamu tidak pernah punya informasi selain kemiringan di satu titik tempatmu berdiri sekarang. Kamu tidak tahu ada jurang dua langkah di depan. Kamu tidak tahu lembah yang kamu masuki adalah cekungan kecil atau dasar sebenarnya. Kebutaan total ini bukan keterbatasan teknis yang nanti diperbaiki. Itu memang kondisi kerjanya, dan seluruh trik optimasi modern lahir dari upaya bertahan hidup di dalamnya.

Gradient descent adalah melangkah berulang kali

Sekarang gabungkan. Kamu punya cara mengukur seberapa salah (loss), dan cara mengetahui arah menurun (gradien). Sisanya tinggal mengulang:

1. hitung tebakan dengan kenop sekarang
2. hitung seberapa salah
3. rasakan kemiringannya
4. geser tiap kenop sedikit ke arah menurun
5. ulangi

Lima baris itu melatih regresi linearmu. Lima baris itu juga melatih GPT. Perbedaannya cuma pada jumlah kenop, ukuran data, dan berapa lama langkah ketiga memakan waktu.

Setiap satu putaran namanya iterasi atau step. Satu putaran penuh melewati seluruh dataset namanya epoch. Itu cuma penamaan, tidak ada ide baru di situ.

Learning rate adalah panjang langkahmu

Gradien memberi tahu arah. Ia tidak memberi tahu seberapa jauh kamu harus melangkah. Itu keputusanmu, dan namanya learning rate.

Terlalu kecil, kamu bergerak semili demi semili dan butuh sejuta iterasi untuk sampai ke tempat yang seharusnya bisa dicapai dalam seribu.

Terlalu besar, kamu melompati lembah dan mendarat di lereng seberang yang posisinya lebih tinggi dari titik awalmu. Iterasi berikutnya melompat balik, lebih tinggi lagi. Loss-mu meledak jadi $\inf$ lalu berubah jadi nan, dan program berhenti berarti.

Kamu sudah punya intuisi ini dari Mekanika, cuma belum menyadarinya.

Permukaan loss model linear berbentuk parabola. Parabola adalah bentuk potensial harmonik $V = (1/2) k x^2$. Jadi parameter modelmu betulan berperilaku seperti massa di ujung pegas. Learning rate berperan seperti kebalikan redaman. Redaman cukup, sistemnya melandai mulus ke titik setimbang. Redaman kurang, sistemnya berosilasi. Redaman kurang parah, amplitudonya membesar tiap siklus dan sistemnya lepas.

Ini bukan kemiripan yang puitis. Ini persamaan yang sama, dan kalau kamu turunkan syarat konvergensi gradient descent untuk loss kuadrat, kamu akan mendapat $\text{learning_rate} < 2/k$ dengan k adalah kelengkungan. Batas kestabilan yang bentuknya persis sama dengan yang kamu kenal dari osilator teredam.

Tanya diri sendiri

Jelaskan tanpa menyebut kata gradien, loss, atau parameter: apa yang sebenarnya dikerjakan komputer saat melatih model?

Kenapa gradient descent butuh tanda minus?

Loss modelmu berhenti di angka 1,35 dan tidak mau turun lagi. Sebutkan tiga kemungkinan penyebab yang saling berbeda secara mendasar.

Kalau learning rate kamu kecilkan sepuluh kali, apakah hasil akhirnya akan lebih baik? Jawab dengan “tergantung” lalu jelaskan tergantung apa.

Kenapa loss di parameter yang benar tidak nol, dan apa yang harus kamu curigai kalau ternyata nol?

Bagian 2. Menghafal lawan memahami

Overfitting

Ada dua murid menghadapi ujian yang sama.

Murid pertama menghafal seratus soal tahun lalu beserta kunci jawabannya. Diberi soal tahun lalu, dia dapat nilai sempurna. Diberi soal baru, dia hancur.

Murid kedua paham konsepnya. Diberi soal tahun lalu, dia dapat 85 karena ada satu dua yang keliru hitung. Diberi soal baru, dia tetap dapat 85.

Model punya penyakit yang persis sama, namanya overfitting.

Cara mendeteksinya cuma satu, dan ini alasan kenapa data selalu dibagi dua. Kamu latih model pada bagian pertama, lalu uji pada bagian kedua yang belum pernah dia lihat. Kalau skor di data latih terus membaik sementara skor di data uji mulai memburuk, model sudah pindah dari memahami ke menghafal.

Titik saat kedua kurva itu berpisah adalah salah satu grafik paling penting di seluruh machine learning. Kamu akan menggambarinya sendiri di Sesi 6.

Di mana analoginya rusak. Murid yang menghafal biasanya sadar dia menghafal. Model tidak. Model yang overfit tidak menunjukkan gejala apa pun dari dalam. Loss latihnya justru kelihatan cantik sekali, lebih cantik dari model yang sehat. Kalau kamu tidak menyisihkan data uji, kamu tidak akan pernah tahu, dan kamu akan mengira modelmu jenius sampai dia dipakai di dunia nyata dan gagal total.

Ini contoh langsung dari prinsip Feynman yang paling saya suka. Orang paling gampang kamu bohongi adalah dirimu sendiri, dan model yang overfit adalah mesin pembohong diri yang sangat efisien.

Kenapa overfitting terjadi

Kamu punya 10 titik data. Kamu pasang polinomial derajat 9.

Polinomial derajat 9 punya 10 koefisien, jadi ia punya kelenturan yang cukup untuk melewati persis 10 titik itu. Loss-nya nol. Sempurna.

Tapi lihat kurvanya di antara titik-titik itu. Dia melonjak liar ke atas dan ke bawah, menempuh rute yang absurd, cuma supaya bisa menyentuh setiap titik. Tanya dia nilai di antara dua titik, jawabannya ngawur.

Model itu tidak mempelajari polanya. Dia mempelajari deraunya. Dan derau, menurut definisinya, tidak berulang. Jadi semua yang dia pelajari dengan susah payah tidak berguna sama sekali di data baru.

Aturan kasarnya: makin banyak kenop dibanding jumlah data, makin gampang model kabur ke menghafal.

Saya harus jujur di sini, karena ini salah satu tempat di mana bidang ini sendiri belum selesai memahami dirinya. Aturan kasar tadi seharusnya berarti model raksasa dengan miliaran parameter yang dilatih pada data terbatas pasti overfit parah. Kenyataannya banyak yang tidak, dan malah membaik setelah melewati titik yang menurut teori klasik seharusnya jadi bencana. Gejalanya punya nama, double descent, tapi nama bukan penjelasan. Penjelasan lengkapnya masih diperdebatkan sampai sekarang. Kalau ada yang menerangkannya ke kamu dengan sangat percaya diri, curigai.

Regularisasi adalah pegas yang menarik kenop pulang

Kalau overfitting datang dari model yang terlalu lentur, obatnya adalah membuat kelenturan itu mahal.

Caranya: tambahkan denda ke loss, sebesar kuadrat semua parameter.

$\text{loss_total} = \text{loss_biasa} + \text{lambda} * \text{jumlah dari } w^2$

Sekarang model punya dua kepentingan yang bertabrakan. Dia mau mencocokkan data, itu menurunkan suku pertama. Tapi dia juga mau menjaga parameternya tetap kecil, itu menurunkan suku kedua. Dia menetap di kompromi.

Sekarang lihat suku itu baik-baik. $\lambda * w^2$. Kamu sudah pernah lihat bentuk ini seumur hidupmu.

Itu energi potensial pegas. Turunkan terhadap w , kamu dapat $2 * \lambda * w$, sebuah gaya pemulih yang sebanding dengan simpangan dan arahnya menuju nol. Itu Hukum Hooke, tidak lebih tidak kurang.

Jadi regularisasi L2 secara harfiah adalah mengikat setiap parameter ke titik nol dengan pegas. λ adalah konstanta pegasnya. Naikkan λ , pegasnya kaku, parameter tertahan dekat nol, model jadi kaku dan mungkin kurang mampu. Turunkan λ , pegasnya lembek, model bebas melenggang ke wilayah menghafal.

Ini bukan analogi. Ini matematika yang sama dengan nama yang berbeda, dan ini contoh pertama dari sesuatu yang akan terus terjadi sepanjang perjalananmu: **kamu sudah menguasai barangnya, kamu cuma belum tahu nama panggilannya di sini.**

Tanya diri sendiri

Model A dapat 99 persen di data latih dan 72 persen di data uji. Model B dapat 85 dan 84. Mana yang kamu pakai, dan kenapa?

Kenapa menambah data biasanya mengurangi overfitting? Jelaskan lewat perbandingan jumlah kenop terhadap jumlah kendala.

Kalau λ kamu setel sangat besar, apa yang terjadi pada model, dan kenapa hasilnya jadi buruk juga?

Kamu punya seratus titik data dan model dengan seribu parameter. Sebutkan tiga cara berbeda menghindari bencana.

Bagian 3. Dari garis lurus ke jaringan saraf

Kenapa garis tidak akan pernah cukup

Bayangkan titik-titik merah tersusun melingkar seperti cincin donat, dan titik-titik biru berkumpul di lubang tengahnya.

Coba pisahkan keduanya dengan satu garis lurus.

Tidak bisa. Bukan sulit, tapi mustahil. Tidak ada garis lurus yang bisa memisahkan lingkaran dari isinya.

Sekarang bagian yang mengejutkan orang. Kalau kamu tumpuk dua lapisan linear, hasilnya tetap linear. Tumpuk sepuluh, tetap linear. $w_2 * (w_1 * x + b_1) + b_2$ bisa disederhanakan jadi $w * x + B$, dan itu tetap satu garis.

Menumpuk penggaris tidak menghasilkan lengkungan. Sebanyak apa pun kamu tumpuk.

Jadi ada satu bahan yang benar-benar hilang.

Fungsi aktivasi adalah tekukannya

Bahan yang hilang itu adalah tekukan. Satu operasi tidak linear yang disisipkan di antara lapisan. Yang paling banyak dipakai namanya ReLU, dan aturannya menyederhanakan ini: kalau angkanya negatif, jadikan nol. Kalau positif, biarkan.

$$\text{relu}(x) = \max(0, x)$$

Itu saja. Bukan penyederhanaan, memang cuma itu isinya.

Dan operasi remeh itu mengubah segalanya. Sekarang tiap neuron punya satu titik patah. Susun ribuan neuron, kamu punya ribuan patahan, dan dengan patahan sebanyak itu kamu bisa membentuk lengkungan apa pun yang kamu mau, sedekat apa pun.

Ada teorema yang menyatakan hal ini secara resmi, namanya universal approximation theorem. Isinya: jaringan dengan satu lapisan tersembunyi yang cukup lebar bisa mendekati fungsi kontinu apa pun sedekat yang kamu minta.

Tapi hati-hati dengan teorema ini, karena ia sering dipakai untuk menipu diri. Teorema itu bilang jaringannya **ada**. Ia tidak bilang gradient descent akan **menemukannya**. Ia juga tidak bilang berapa lebar yang dibutuhkan, dan jawabannya bisa saja lebar yang tidak muat di alam semesta. Jarak antara “ada solusinya” dan “saya bisa mendapatkannya” adalah jarak antara matematika dan rekayasa, dan seluruh kesulitan praktis deep learning hidup di jarak itu.

Jaringan saraf adalah tumpukan yang ditekuk

Sekarang kamu punya semua bahannya, dan resepnya jadi pendek.

Kalikan matriks, tekuk. Kalikan matriks lagi, tekuk lagi. Ulangi sebanyak yang kamu mau. Di ujung, keluarkan jawaban.

```
x -> [kali matriks] -> [tekuk] -> [kali matriks] -> [tekuk] -> [kali matriks] -> jawaban
```

Itu jaringan saraf. Betulan itu.

Kata “neuron” datang dari inspirasi biologis di tahun 1940-an, dan sekarang lebih banyak menyesatkan daripada membantu. Yang disebut neuron adalah satu baris dalam matriks bobot, ditambah satu bilangan bias, lalu ditekuk. Ia tidak menyala, tidak berkomunikasi, tidak melakukan apa pun yang dilakukan sel saraf. Ia sebuah baris dalam matriks.

Saya menekankan ini karena kata yang salah bisa menghalangi pemahamanmu bertahun-tahun. Kalau kamu terus membayangkan sel otak, kamu akan terus merasa ada keajaiban yang belum kamu pahami. Tidak ada. Yang ada perkalian matriks dan tekukan.

Backpropagation, dan kenapa arahnya mundur

Ini bagian yang orang anggap sulit. Sebenarnya tidak, asal kamu bertanya dengan urutan yang benar.

Kamu punya jaringan dengan satu juta kenop. Kamu punya satu angka loss di ujung. Kamu perlu tahu, untuk **setiap satu** dari sejuta kenop itu, seberapa besar pengaruhnya terhadap loss.

Cara paling lugu: goyangkan kenop pertama sedikit, jalankan ulang seluruh jaringan, lihat perubahan loss. Lalu kenop kedua. Lalu kenop ketiga.

Sejuta kali jalan penuh, untuk satu langkah pelatihan. Kalau satu jalan makan sepersepuluh detik, satu langkah butuh lebih dari sehari. Kamu butuh puluhan ribu langkah. Selesai sebelum mulai.

Sekarang pertanyaannya jadi jelas. Bukan “bagaimana cara menghitung turunan”, kamu sudah bisa itu sejak Fisika Matematika. Pertanyaannya adalah **bagaimana mendapat sejuta turunan dengan harga satu kali jalan.**

Jawabannya: balik arahnya.

Mulai dari loss di ujung. Turunan loss terhadap dirinya sendiri adalah 1, itu titik berangkatnya. Lalu mundur satu operasi ke belakang, dan tanyakan: berapa turunan loss terhadap masukan operasi ini? Aturan rantai menjawabnya, dan jawabannya cuma butuh dua hal, yaitu turunan lokal operasi itu dan turunan yang sudah kamu bawa dari belakang.

Kalikan, oper ke belakang, ulangi.

Satu kali jalan mundur, dan semua sejuta turunan sudah di tangan.

Analogi yang saya suka untuk bagian “kenapa mundur”. Sebuah proyek gagal. Kamu mau tahu kontribusi tiap orang dari seribu orang terhadap kegagalannya. Cara maju: tanya satu per satu “kalau kamu tidak ada, hasilnya beda tidak?” Seribu wawancara. Cara mundur: mulai dari kegagalan akhir, bagi tanggung jawab ke beberapa kepala divisi, tiap kepala divisi membagi jatah tanggung jawabnya ke bawahannya, terus turun sampai orang terakhir. Satu putaran, semua orang dapat angkanya.

Backpropagation adalah itu. Bukan matematika baru. Aturan rantai yang kamu sudah kuasai, dijalankan dengan urutan yang efisien.

Di mana analoginya rusak. Pembagian tanggung jawab manusia bersifat subjektif dan bisa didebat. Pembagian di backprop adalah turunan parsial, dan nilainya tunggal, pasti, serta bisa diverifikasi. Kamu akan memverifikasinya sendiri dengan beda hingga dan mendapat kecocokan sampai di bawah $1e-6$. Kalau kecocokannya meleset, bukan alamnya yang salah, kodemu yang salah.

Autograd adalah buku catatan

Menurunkan gradien dengan tangan untuk model dua parameter itu latihan yang sehat. Untuk model sepuluh lapis, itu penyiksaan, dan kamu pasti salah tanda di suatu tempat.

Jadi kita suruh komputer yang mencatat.

Idenya sederhana. Tiap kali sebuah angka lahir dari operasi, ia mencatat dua hal di buku: siapa induknya, dan operasi apa yang melahirkannya. Lakukan seluruh perhitungan maju seperti biasa. Di akhir, kamu punya catatan lengkap silsilah setiap angka.

Sekarang telusuri buku itu dari belakang, terapkan aturan rantai di tiap langkah, dan gradien mengalir pulang sendiri.

Itu isi `loss.backward()`. Tidak ada sihir di dalamnya. Kamu akan menulis versimu sendiri di Modul 1, sekitar 150 baris, dan setelah malam itu PyTorch berhenti terasa gaib untuk selamanya.

Ini menurut saya momen paling berharga di seluruh silabusmu. Bukan karena kamu akan memakai autograd buatanmu (kamu tidak akan, PyTorch jauh lebih cepat), tapi karena setelah menulisnya kamu tidak akan pernah lagi berhadapan dengan kotak hitam di lapisan paling dasar.

Tanya diri sendiri

Kenapa menumpuk sepuluh lapisan linear tanpa aktivasi tetap sama saja dengan satu lapisan? Buktikan dengan aljabar untuk dua lapisan.

Kenapa backprop berjalan mundur, bukan maju? Jawab dengan hitungan biaya, bukan dengan “karena begitu rumusnya”.

Apa isi buku catatan autograd, dan kapan ia ditulis?

Gradient check kamu meleset di angka desimal ketiga. Sebutkan dua kemungkinan penyebab yang berbeda jenis.

Kenapa kata “neuron” lebih menyesatkan daripada membantu?

Bagian 4. Mengubah dunia jadi angka

Mesin tidak mengerti kata, ia mengerti tempat

Ini pergeseran cara pikir yang paling penting di sisa dokumen ini.

Komputer tidak bisa mengolah kata “kucing”. Ia butuh angka. Tapi memberi nomor sembarangan (kucing = 1, anjing = 2, meja = 3) menciptakan kebohongan, karena sekarang komputer mengira anjing berada di tengah antara kucing dan meja.

Solusinya: jangan beri nomor, beri **koordinat**.

Bayangkan peta kota, tapi yang dipetakan makna. Kucing dan anjing berdekatan karena sama-sama hewan peliharaan. Meja jauh dari keduanya. Kursi dekat meja. Singa agak dekat kucing tapi tidak terlalu.

Koordinat itu namanya embedding. Bedanya dengan peta kota cuma jumlah sumbunya. Peta punya dua, embedding punya 300 atau 768 atau 4096.

Dan begitu makna jadi tempat, kemiripan jadi jarak. Itu satu kalimat, tapi seluruh NLP modern, seluruh pengenalan wajah, dan seluruh mesin rekomendasi berdiri di atasnya.

Di mana analoginya rusak. Di peta kota, sumbunya punya arti yang bisa disebut, yaitu utara dan timur. Di embedding, sumbunya tidak punya nama dan biasanya tidak berarti apa-apa satu per satu. Kamu mungkin pernah dengar contoh terkenal raja - pria + wanita = ratu. Itu memang bekerja, kadang. Ia juga sering gagal, dan orang yang mengutipnya jarang menyebutkan bagian gagalnya. Perlakukan sebagai gejala menarik, bukan sebagai hukum.

Jarak, atau lebih tepatnya sudut

Kalau makna adalah tempat, mengukur kemiripan berarti mengukur jarak. Tapi dalam praktik yang dipakai bukan jarak, melainkan **sudut**.

Alasannya: panjang vektor sering membawa informasi yang tidak kamu inginkan, misalnya seberapa sering kata itu muncul. Arahnya yang membawa makna. Jadi normalkan semua vektor jadi panjang satu, lalu bandingkan sudutnya. Nilainya keluar lewat hasil kali dalam, dan namanya cosine similarity.

Sekarang buka catatan Fisika Kuantum kamu.

Kamu menghitung $\langle \psi | \phi \rangle$ untuk mengetahui seberapa besar tumpang tindih dua keadaan. Keadaan dinormalkan jadi norma satu. Hasil kali dalam yang mendekati satu berarti keadaannya hampir sama, mendekati nol berarti ortogonal alias tidak berhubungan.

Itu operasi yang **sama persis**. Vektor ternormalkan, hasil kali dalam, tafsirkan sebagai tumpang tindih. Yang berubah cuma nama ruangnya.

Jadi saat orang bilang “embedding hidup di ruang berdimensi tinggi dan kemiripan diukur dengan cosine similarity”, yang mereka maksud adalah hal yang kamu sudah kerjakan berbulan-bulan di kelas Kuantum, dengan kosakata yang berbeda.

Softmax mengubah skor jadi peluang

Modelmu mengeluarkan beberapa angka mentah, satu per kelas. Misalnya $[2.1, 0.5, -1.3]$ untuk tiga kemungkinan perintah.

Angka mentah itu tidak enak dipakai. Kamu mau peluang: angka positif yang jumlahnya pas satu. Softmax mengerjakannya dalam dua langkah. Eksponenkan semuanya, lalu bagi dengan totalnya.

$$\text{softmax}(z_i) = \exp(z_i) / \text{jumlah dari } \exp(z_j)$$

Eksponen memastikan semua jadi positif, dan membuat skor tinggi jadi jauh lebih dominan daripada selisih aslinya. Pembagian memastikan totalnya satu.

Sekarang tambahkan satu parameter, bagi eksponennya dengan T :

$$\text{softmax}(z_i) = \exp(z_i / T) / \text{jumlah dari } \exp(z_j / T)$$

Buka catatan Termodinamika kamu, cari distribusi Boltzmann:

$$P(E_i) = \exp(-E_i / kT) / Z$$

Bentuknya sama. Penyebutnya bahkan punya peran yang sama, dan di fisika namanya fungsi partisi.

Ini bukan kebetulan dan bukan analogi longgar. Parameter temperature di LLM memang dinamai dari suhu termodinamika, dan ia berperilaku seperti suhu. T rendah berarti sistem beku, model hampir selalu memilih kata dengan skor tertinggi, keluarannya kaku dan berulang. T tinggi berarti sistem panas, peluang tersebar lebih merata, model memilih kata yang lebih tidak terduga, keluarannya kreatif atau ngaco tergantung seberapa panas.

Saat kamu nanti menyetel $\text{temperature}=0.7$ di Ollama pada Modul 6, kamu sedang mengatur suhu sebuah sistem statistik. Secara harfiah, bukan secara kiasan.

Cross-entropy mengukur keterkejutan

Loss untuk klasifikasi bukan MSE. Ia cross-entropy, dan cara termudah memahaminya adalah lewat kata “kejutan”.

Model bilang “saya 99 persen yakin ini kucing”. Ternyata kucing. Kejutannya kecil, jadi loss-nya kecil.

Model bilang “saya 99 persen yakin ini kucing”. Ternyata anjing. Kejutannya besar sekali, dan loss-nya besar sekali.

Model bilang “saya 50 persen yakin”. Apa pun jawabannya, kejutannya sedang.

Rumusnya cuma minus logaritma peluang yang diberikan model kepada jawaban yang benar:

$$\text{loss} = -\log(p_{\text{benar}})$$

Coba masukkan angka. $p = 1$ memberi loss nol, tidak terkejut sama sekali. $p = 0.5$ memberi 0,69. $p = 0.01$ memberi 4,6. p mendekati nol memberi loss yang meledak ke tak hingga, dan itu hukuman untuk percaya diri yang salah besar.

Sekarang lihat namanya. Entropi. Kamu sudah punya besaran itu di Termodinamika, dan definisinya di sana adalah $S = -k \cdot \sum p \log p$. Bentuk yang sama. Yang diukur juga hal yang sama, yaitu seberapa banyak ketidaktahuan yang terkandung dalam sebuah distribusi peluang.

Claude Shannon meminjam nama itu dari termodinamika di tahun 1948, dan dia meminjamnya karena matematikanya memang identik, bukan karena namanya terdengar keren.

Tanya diri sendiri

Kenapa memberi nomor urut ke kata adalah ide buruk?

Kenapa cosine similarity lebih sering dipakai daripada jarak Euclid?

Tuliskan padanan $\langle \psi | \phi \rangle$ di dunia machine learning, dan jelaskan apa yang diukur keduanya.

Kalau temperature disetel nol, apa yang terjadi? Kaitkan dengan sistem fisis pada suhu nol mutlak.

Kenapa cross-entropy menghukum “yakin tapi salah” jauh lebih keras daripada “ragu dan salah”? Apakah itu perilaku yang kamu inginkan?

Bagian 5. Suara

Sampling, dan roda kereta yang berputar mundur

Suara adalah tekanan udara yang berubah terhadap waktu. Kurva mulus, tanpa putus.

Komputer tidak bisa menyimpan kurva mulus. Ia mengambil cuplikan pada selang tetap, misalnya 16.000 kali per detik, dan menyimpan daftar angka.

Pertanyaannya: berapa cepat harus mencuplik supaya tidak ada yang hilang?

Kamu sudah tahu jawabannya dari kelas Gelombang. Minimal dua kali frekuensi tertinggi yang ada di sinyal. Itu Nyquist.

Dan kamu sudah pernah melihat akibat melanggarnya, di film. Roda kereta yang berputar cepat tampak berputar pelan, berhenti, bahkan berputar mundur. Kamera mencuplik 24 kali per detik, roda berputar lebih cepat dari itu, dan otakmu merekonstruksi frekuensi palsu yang lebih rendah.

Itu aliasing. Frekuensi tinggi yang tercuplik terlalu jarang menyamar jadi frekuensi rendah, dan begitu ia menyamar, tidak ada cara mengembalikannya. Informasinya tidak rusak, ia hilang.

Itu sebabnya sistem audio memasang low-pass filter **sebelum** ADC, bukan sesudah. Sesudah sudah terlambat.

Fourier, atau memisahkan akor jadi nada

Tekan tiga tuts piano bersamaan. Yang sampai ke telingamu satu gelombang tekanan tunggal, satu kurva rumit.

Tapi telingamu bisa mendengar tiga nada terpisah.

Telingamu melakukan transformasi Fourier. Koklea di dalam telinga secara fisik terurai sepanjang panjangnya berdasarkan frekuensi, dengan bagian pangkal peka ke nada tinggi dan bagian ujung ke nada rendah. Ia perangkat keras analisis frekuensi yang dibuat dari daging.

Transformasi Fourier adalah versi matematikanya. Masukkan kurva rumit, keluar daftar “seberapa banyak frekuensi ini ada di dalamnya” untuk tiap frekuensi.

Ada satu masalah. Fourier polos memberi tahu frekuensi apa saja yang ada di seluruh rekaman, tapi tidak memberi tahu kapan. Untuk suara ucapan, kapan justru segalanya. Kata “satu” dan “tuas” bisa punya kandungan frekuensi yang mirip dan artinya jauh berbeda.

Jadi potong dulu. Iris sinyal jadi kepingan pendek sekitar 25 milidetik, transformasikan tiap keping, susun hasilnya berjajar.

Hasilnya spektrogram. Sumbu datar waktu, sumbu tegak frekuensi, warna menyatakan kekuatan. Itu partitur dari suaramu.

Dan begitu suara sudah jadi gambar, semua alat pengolah gambar bisa dipakai. Itu triknya, dan itu alasan pengenalan suara modern memakai arsitektur yang sama dengan pengenalan gambar.

Windowing, yang di kelas DSP terasa seperti detail teknis membosankan, sekarang punya alasan yang bisa kamu rasakan. Memotong sinyal secara mendadak menciptakan ujung tajam palsu, dan ujung tajam palsu itu melahirkan frekuensi palsu yang mengotori spektrogram. Jendela Hann melandaikan ujungnya supaya potongan itu tidak berbohong.

Konvolusi adalah stempel yang digeser

Kamu punya gambar, dan kamu ingin tahu di mana ada tepi tegak.

Buat kotak kecil 3 kali 3 berisi angka yang bernilai besar kalau menemukan tepi tegak. Tempelkan kotak itu di pojok kiri atas gambar, kalikan pasangan piksel dengan angka di kotak, jumlahkan, catat hasilnya. Geser satu piksel. Ulangi. Sampai seluruh gambar tersapu.

Hasilnya peta yang menyala di setiap lokasi yang punya tepi tegak.

Itu konvolusi. Satu stempel kecil, digeser ke seluruh permukaan.

Kamu sudah kenal operasi ini dari Fisika Matematika III, cuma dengan cerita yang berbeda. Di sana konvolusi adalah cara sistem linear menanggapi masukan, dengan mengaburkan masukan itu memakai fungsi tanggapnya. Detektor yang tidak sempurna mengaburkan sinyal aslinya, dan pengaburan itu adalah konvolusi dengan fungsi sebar titik.

Sama persis. Yang berbeda cuma niatnya. Di fisika kamu biasanya ingin **membatalkan** konvolusi untuk memulihkan sinyal asli. Di CNN kamu ingin **mempelajari** stempelnya, supaya ia mendeteksi apa pun yang berguna.

Dan teorema konvolusi yang kamu hafal, bahwa konvolusi di ranah waktu sama dengan perkalian di ranah frekuensi, adalah alasan konvolusi besar dihitung lewat FFT dan bukan lewat penjumlahan langsung.

CNN dan kenapa satu stempel dipakai berulang

Jaringan biasa memperlakukan tiap piksel sebagai masukan terpisah dengan bobotnya sendiri. Untuk gambar 200 kali 200 itu 40.000 masukan, dan lapisan pertama saja sudah butuh jutaan bobot.

Lebih buruk lagi, jaringan itu harus belajar “seperti apa bentuk tepi” secara terpisah untuk setiap lokasi. Kalau dilatih dengan kucing di pojok kiri, ia tidak akan mengenali kucing di pojok kanan.

CNN memakai dua ide sederhana untuk memperbaiki keduanya.

Bobot yang sama dipakai di semua lokasi. Satu stempel, digeser ke seluruh gambar. Tepi adalah tepi di mana pun ia berada, jadi tidak ada gunanya mempelajari ulang. Ini memangkas jumlah bobot dari jutaan jadi puluhan, dan sekaligus membuat jaringan mengenali objek di posisi mana pun.

Susun berlapis. Lapisan pertama menemukan tepi. Lapisan kedua menggabungkan tepi jadi sudut dan lengkungan. Lapisan ketiga menggabungkan itu jadi bagian, misalnya mata atau roda. Lapisan berikutnya jadi objek utuh.

Yang mengesankan, hierarki ini tidak diprogram siapa pun. Ia muncul sendiri dari pelatihan, dan orang baru menyadarinya setelah memvisualisasikan filter yang terlatih. Lapisan pertama CNN yang dilatih pada foto hampir selalu berakhir jadi detektor tepi, mirip dengan yang ditemukan Hubel dan Wiesel di korteks visual kucing pada tahun 1959.

Saya harus jujur soal batas pemahaman di sini juga. Kita bisa melihat apa yang dilakukan lapisan awal karena filternya kecil dan bisa digambar. Untuk lapisan dalam, penjelasan macam “neuron ini mendeteksi konsep anjing” sebagian besar adalah cerita yang kita karang setelah melihat gambar yang membuatnya menyala paling kuat. Bidang yang mempelajari ini serius, interpretability, masih sangat muda.

Tanya diri sendiri

Kenapa low-pass filter harus dipasang sebelum ADC dan bukan sesudah?

Kenapa Fourier polos tidak cukup untuk suara ucapan?

Kenapa windowing dibutuhkan? Jelaskan lewat apa yang terjadi kalau tidak dipakai.

Apa persamaan dan perbedaan konvolusi di CNN dengan konvolusi di kelas Fisika Matematika?
Kenapa berbagi bobot memecahkan dua masalah sekaligus? Sebutkan keduanya.

Bagian 6. Wajah

Jarak sebagai identitas

Cara yang naif untuk mengenali wajah: simpan fotomu, lalu bandingkan foto baru piksel demi piksel.

Ini gagal total. Ganti pencahayaan, semua piksel berubah. Miringkan kepala lima derajat, semua piksel berubah. Foto dua orang kembar di ruangan yang sama akan lebih mirip secara piksel daripada dua fotomu sendiri di ruangan berbeda.

Yang bekerja adalah ini: latih jaringan untuk memetakan wajah ke vektor, dengan satu syarat pelatihan yang bunyinya begini.

Dua foto orang yang sama harus menghasilkan vektor yang berdekatan. Dua foto orang berbeda harus menghasilkan vektor yang berjauhan.

Perhatikan apa yang **tidak** ada di syarat itu. Tidak ada perintah “cari hidung”. Tidak ada daftar ciri wajah. Tidak ada yang memberi tahu jaringan apa yang harus dilihat. Yang diberikan cuma aturan tentang jarak, dan jaringan menemukan sendiri fitur apa yang membuat aturan itu terpenuhi.

Namanya metric learning. Yang dipelajari bukan jawabannya, melainkan **ukuran jaraknya**.

Hasil akhirnya: tiap wajah jadi satu titik di permukaan bola berdimensi 512. Wajahmu punya wilayah kecil di bola itu. Mengenali wajah berarti mengukur sudut, dan itu operasi yang sama dengan Bagian 4.

Ini juga alasan sistem ini bisa mengenali orang yang belum pernah ada di data latihnya. Ia tidak menghafal orang. Ia mempelajari cara mengukur, dan alat ukur bekerja pada apa pun yang kamu ukur.

Ambang adalah keputusan, bukan hitungan

Dua vektor punya sudut 23 derajat. Orang yang sama atau bukan?

Tidak ada jawaban yang bisa dihitung. Yang ada garis yang harus kamu tarik.

Tarik longgar, misalnya terima sampai 40 derajat, dan orang asing bisa masuk ke sesimu. Namanya false accept.

Tarik ketat, misalnya cuma sampai 10 derajat, dan kamu sendiri akan ditolak saat kurang tidur atau lampunya redup. Namanya false reject.

Dua kesalahan itu bergerak berlawanan. Menurunkan yang satu selalu menaikkan yang lain. Tidak ada setelan yang membuat keduanya nol, dan mengejar itu adalah membohongi diri sendiri.

Yang ada cuma pertanyaan: kesalahan mana yang lebih mahal buat kamu?

Untuk membuka aplikasi catatan, false reject lebih menyebalkan daripada false accept, jadi longgarkan. Untuk menyetujui perintah $rm -rf$, false accept adalah bencana dan false reject cuma merepotkan, jadi ketatkan habis.

Kurva ROC adalah grafik yang memetakan seluruh pertukaran ini. Kamu sapu ambang dari longgar ke ketat, plot pasangan kedua tingkat kesalahan, dan kamu dapat kurva yang menunjukkan semua pilihan yang tersedia. Kurva itu tidak memilihkan untukmu. Ia cuma menunjukkan harga tiap pilihan.

Ini salah satu tempat pertama kamu akan menemui kenyataan bahwa membangun sistem melibatkan penghakiman, bukan cuma perhitungan. Model bisa dilatih. Ambang harus diputuskan, oleh kamu, dan kamu harus bisa menjelaskan kenapa angkanya di situ.

Tanya diri sendiri

Kenapa membandingkan foto piksel demi piksel gagal?

Bagaimana jaringan bisa mengenali wajah yang tidak pernah ada di data latihnya?

Kalau kamu turunkan false accept rate jadi nol, apa yang pasti terjadi pada sisi lainnya?

Untuk SYNESIS, di mana kamu akan menaruh ambangnya, dan apa alasan yang bisa kamu pertahankan?

Bagian 7. Bahasa

Token, potongan yang lebih kecil dari kata

Model tidak bekerja per huruf, karena terlalu lambat. Tidak juga per kata, karena kosakatanya jadi tak terbatas dan kata baru bikin macet.

Jalan tengahnya token, potongan yang sering muncul. Kata umum jadi satu token. Kata jarang dipecah jadi beberapa. Nama orang mungkin jadi empat potong.

Itu sebabnya LLM kadang gagal pada tugas yang tampak sepele, misalnya menghitung huruf dalam sebuah kata. Ia tidak melihat huruf. Ia melihat potongan, dan hurufnya tidak pernah sampai ke matanya.

Bukan kebodohan model. Keterbatasan matanya.

Attention, atau mencari di perpustakaan

Ini konsep paling penting di Bagian 7, dan sebenarnya bisa dipahami tanpa satu pun rumus.

Baca kalimat ini: “Gelas itu jatuh ke lantai karena **ia** licin.”

Untuk mengerti kalimat itu, kamu harus tahu “ia” merujuk ke apa. Gelasnya? Lantainya? Kamu melihat balik ke kata-kata sebelumnya, memberi perhatian lebih ke sebagian dan mengabaikan sisanya, lalu memutuskan.

Kamu baru saja melakukan attention. Model mengerjakan hal yang sama, dengan cara yang bisa dihitung.

Bayangkan perpustakaan. Kamu datang dengan sebuah pertanyaan. Setiap buku punya judul di punggungnya dan isi di dalamnya. Kamu bandingkan pertanyaanmu dengan semua judul, memberi tiap buku skor kecocokan, lalu meracik jawabanmu dari isi semua buku dengan takaran sesuai skornya. Buku yang judulnya sangat cocok menyumbang banyak. Yang tidak cocok menyumbang hampir tidak ada.

Tiga benda di cerita itu punya nama di transformer.

Pertanyaanmu adalah **query**. Judul di punggung buku adalah **key**. Isi buku adalah **value**.

Dan tiap kata dalam kalimat memainkan ketiga peran sekaligus. Ia bertanya (query), ia menawarkan dirinya untuk ditemukan (key), dan ia menyediakan isinya untuk diambil (value).

Rumusnya sekarang jadi bisa dibaca:

$$\text{attention} = \text{softmax}(Q \text{ dikali } K \text{ transpos} / \text{akar } d) \text{ dikali } V$$

Q dikali K transpos adalah mencocokkan tiap pertanyaan dengan tiap judul, dan itu cuma hasil kali dalam, alat ukur kemiripan dari Bagian 4 lagi. softmax mengubah skor kecocokan jadi takaran yang jumlahnya satu. Mengalikan dengan V adalah meracik isi sesuai takaran. Pembagi akar d adalah penyesuaian skala supaya softmax tidak jadi terlalu tajam saat dimensinya besar.

Setiap bagian rumus itu punya padanan di cerita perpustakaan, dan tidak ada bagian yang tersisa tanpa penjelasan.

Di mana analoginya rusak. Di perpustakaan, buku sudah ada dan judulnya sudah tertulis. Di transformer, query, key, dan value ketiganya dihasilkan dari kata yang sama lewat tiga matriks bobot berbeda, dan ketiga matriks itu **dipelajari**. Jadi model tidak cuma mencari di perpustakaan. Ia sekaligus belajar cara menulis judul buku yang bagus dan cara merumuskan pertanyaan yang bagus, keduanya sambil berjalan.

Multi-head, beberapa pencari sekaligus

Satu kalimat perlu dipahami dari beberapa sudut sekaligus. Siapa pelakunya, kata mana merujuk ke mana, mana kata sifat untuk mana kata benda.

Satu attention head cuma bisa mengejar satu jenis hubungan pada satu waktu. Jadi pasang delapan atau enam belas sekaligus, masing-masing dengan matriks bobotnya sendiri, lalu gabungkan hasilnya.

Delapan orang masuk ke perpustakaan yang sama secara bersamaan, masing-masing mencari hal yang berbeda, lalu duduk bersama dan menggabungkan temuan.

Yang penting: tidak ada yang menugaskan head nomor tiga untuk mengurus kata ganti. Pembagian tugas itu muncul sendiri dari pelatihan. Dan seperti yang saya bilang di bagian CNN, cerita kita tentang “head ini mengurus X ” sebagian besar adalah tafsir setelah kejadian, bukan rancangan.

Positional encoding, karena attention buta urutan

Ada masalah serius dengan attention. Ia memandang semua kata sekaligus, tanpa urutan bawaan.

Bagi attention polos, “anjing menggigit pria” dan “pria menggigit anjing” adalah kumpulan kata yang sama persis. Ia tidak punya cara membedakannya.

Jadi urutannya harus dicap ke dalam vektornya. Tiap posisi diberi tanda pengenal unik yang ditambahkan ke embedding katanya.

Yang menarik cara membuat tandanya. Bukan sekadar 1, 2, 3, tapi kombinasi nilai sinus dan kosinus pada banyak frekuensi berbeda.

Buka catatan Fisika Matematika kamu, bab deret Fourier.

Itu basis Fourier. Betulan itu, tanpa modifikasi berarti.

Kenapa pilih itu? Bayangkan jam dengan banyak jarum, masing-masing berputar dengan kecepatan berbeda. Jarum tercepat berputar tiap detik, yang paling lambat berputar tiap tahun. Lihat posisi semua jarum sekaligus, dan kamu bisa menentukan waktu secara unik pada rentang yang sangat lebar sekaligus tetap peka pada perbedaan sekecil satu detik.

Positional encoding melakukan itu untuk posisi. Frekuensi tinggi membedakan kata yang bersebelahan, frekuensi rendah membedakan awal dan akhir dokumen. Bonusnya, dengan basis ini pergeseran posisi jadi bisa dinyatakan sebagai rotasi, sehingga model relatif gampang mempelajari hubungan macam “tiga kata sebelum ini”.

Transformer, semuanya digabung

Sekarang bahannya lengkap, dan resep transformer jadi pendek.

1. ubah token jadi vektor
2. tambahkan penanda posisi
3. ulangi N kali:
 - attention: tiap kata melihat kata lain dan mengambil yang ia butuhkan
 - jaringan biasa: tiap kata mengolah hasilnya sendiri
4. ubah vektor akhir jadi peluang untuk token berikutnya

Itu GPT. Yang membedakan GPT-4 dari mini-GPT yang akan kamu latih di Modul 5 cuma N yang lebih besar, vektor yang lebih lebar, data yang lebih banyak, dan uang listrik yang lebih besar.

Bukan ide yang lebih pintar. Skala yang lebih besar dari ide yang sama.

Saya menekankan ini bukan untuk mengecilkan GPT-4. Membuat ide sederhana bekerja pada skala itu adalah pencapaian rekayasa yang luar biasa. Saya menekankan ini supaya kamu tidak menganggap ada rahasia yang disembunyikan. Tidak ada. Arsitekturnya terbuka, kamu akan membangunnya sendiri, dan setelah itu kamu bisa membaca makalah model terbaru dan mengerti isinya.

Kenapa LLM bisa “mengerti”

Saya taruh tanda kutip di situ dengan sengaja, dan sekarang saya jelaskan kenapa.

Selama pelatihan, LLM cuma mengerjakan satu tugas: tebak token berikutnya. Berulang, triliunan kali, pada teks yang sangat banyak.

Untuk jadi sangat jago menebak kata berikutnya, ternyata kamu terpaksa mempelajari banyak hal lain. Tata bahasa, karena tebakan yang melanggar tata bahasa hampir selalu salah. Fakta, karena “ibu kota Prancis adalah ...” punya satu jawaban yang jauh lebih mungkin. Gaya, nalar sederhana, struktur argumen, semuanya karena semuanya membantu menebak.

Kemampuan itu bukan diprogram. Ia jatuh sebagai efek samping dari tekanan untuk menebak dengan baik.

Sekarang bagian jujurinya. Apakah itu “pemahaman”? Saya tidak tahu, dan saya tidak percaya siapa pun yang mengaku tahu dengan pasti.

Yang bisa saya katakan dengan yakin cuma ini. Mekanismenya adalah prediksi statistik atas token. Perilaku yang muncul di beberapa tugas menyerupai penalaran. Kedua kalimat itu sama-sama benar, dan lompatan dari kalimat pertama ke kesimpulan filosofis apa pun adalah lompatan yang belum ada yang berhasil membuktikan.

Berhati-hatilah pada dua kubu yang sama-sama terlalu percaya diri. Yang bilang “cuma auto-complete canggih” mengabaikan bahwa untuk melakukan autocompleteness sebaik itu, sesuatu yang tidak sepele harus terjadi di dalamnya. Yang bilang “ia berpikir seperti manusia” mengarang mekanisme yang tidak ada buktinya. Posisi jujurinya ada di tengah dan terasa tidak memuaskan, dan rasa tidak puas itu adalah harga kejujuran.

Tanya diri sendiri

Kenapa LLM sering salah menghitung jumlah huruf dalam sebuah kata?

Jelaskan query, key, dan value tanpa memakai kata query, key, atau value.

Kenapa attention butuh positional encoding, padahal RNN tidak?

Kenapa positional encoding memakai sinus alih-alih nomor urut biasa?

Apa yang benar-benar dipelajari LLM selama pelatihan, dan kemampuan apa yang muncul sebagai efek samping?

Bagian 8. Merakit jadi asisten

Agent loop adalah termostat

Kamu sudah punya kerangkanya dari Otomasi Sistem Fisik.

Termostat mengukur suhu, membandingkan dengan target, menyalakan atau mematikan pemanas, lalu mengukur lagi. Lingkaran tertutup dengan umpan balik.

SYNOPSIS berjalan dengan lingkaran yang sama.

dengar -> tangkap suara, ubah jadi teks

pahami -> teks apa maksudnya

putuskan-> tool mana yang dipanggil, dengan argumen apa

lakukan -> jalankan

laporkan-> ucapkan hasilnya

ulangi

Kalau kamu bisa menggambar diagram blok sistem kendali, kamu bisa menggambar SYNESIS. Ini bukan bidang baru buat kamu, cuma isi kotak-kotaknya yang berbeda.

Kenapa SYNESIS tidak butuh LLM sampai Modul 6

Ini keputusan rancangan yang perlu kamu pahami alasannya, karena ia menghemat berbulan-bulan.

Perintah harianmu ke asisten sebenarnya sangat sedikit ragamnya. Buka file, cari file, baca log, cek baterai, matikan wifi. Mungkin tiga puluh jenis, dan setiap jenis punya beberapa cara pengucapan.

Menyuruh LLM empat miliar parameter menangani itu seperti memakai teleskop untuk membaca koran. Lambat, boros, dan malah lebih sering salah karena LLM kadang mengarang tool yang tidak ada.

Classifier kecil yang kamu latih sendiri dari 500 contoh menyelesaikan sebagian besar perintah harian dalam waktu di bawah 10 milidetik, dan hasilnya bisa kamu prediksi.

Jadi susunannya begini. Classifier menangani yang biasa. LLM dipanggil cuma untuk permintaan terbuka yang tidak tertangkap classifier.

Ini juga contoh bagus untuk kebiasaan yang perlu kamu bawa seumur karier. Pertanyaan pertama bukan “model apa yang saya pakai”. Pertanyaan pertama adalah “apakah saya butuh model sama sekali di sini”. Sering jawabannya tidak, dan menjawabnya dengan jujur akan membedakan kamu dari orang yang memasang LLM ke setiap masalah karena semua orang melakukannya.

Tool calling, dan rel kereta bernama grammar

Supaya model bisa menjalankan sesuatu, ia harus mengeluarkan perintah dalam format yang bisa dibaca program. Biasanya JSON.

```
{"tool": "buka_file", "argumen": {"path": "S:/laporan.pdf"}}
```

Masalahnya, model kecil sering menulis JSON yang rusak. Kurung tidak ditutup, koma nyasar, nama tool dikarang sendiri.

Solusinya bukan memohon lewat prompt. Solusinya membatasi apa yang boleh keluar.

Di tiap langkah, model sebenarnya memilih token berikutnya dari daftar peluang. Grammar bekerja dengan mencoret token yang akan merusak format, sebelum pemilihan terjadi. Kalau tata bahasanya sedang menunggu tanda kutip, semua token selain tanda kutip diberi peluang nol.

Bayangkan rel kereta. Kereta boleh memilih arah di setiap persimpangan, tapi ia tidak bisa keluar rel, karena secara fisik tidak ada jalan ke sana.

Hasilnya JSON yang valid seratus persen, dijamin oleh konstruksinya, bukan oleh harapan.

Lapisan aman, ditulis sebelum dibutuhkan

SYNESIS akan punya akses ke berkasmu. Itu memang tujuannya sejak awal. Tapi itu juga berarti satu salah tafsir bisa menghapus sesuatu yang tidak ada cadangannya.

Aturan yang saya sarankan kamu pegang: **lapisan aman ditulis sebelum tool yang berbahaya, bukan sesudah.**

Kalau kamu tunda, kamu akan sedang senang karena sistemnya baru saja bisa jalan, dan menunda lagi terasa wajar. Lalu suatu malam kamu bereksperimen dengan perintah baru dan sesuatu terhapus.

Isi minimalnya: daftar operasi yang wajib konfirmasi manusia, daftar folder yang tidak boleh disentuh sama sekali, batas jumlah operasi per menit, dan satu tombol mati yang bisa kamu tekan tanpa berpikir.

Ini juga tempat pengenalan wajah dari Modul 4 mendapat peran nyatanya. Operasi biasa jalan begitu saja. Operasi yang bisa menghancurkan sesuatu meminta wajahmu dulu.

Tanya diri sendiri

Gambarkan lingkaran SYNESIS sebagai diagram blok sistem kendali. Mana sensornya, mana aktuatornya, mana umpan baliknya?

Kenapa intent classifier lebih baik daripada LLM untuk perintah harian? Beri tiga alasan yang berbeda jenisnya.

Kenapa grammar lebih andal daripada menulis prompt yang memohon JSON valid?

Sebutkan tiga operasi yang harus selalu minta konfirmasi, dan alasan spesifik masing-masing.

Bagian 9. Kamus fisika ke machine learning

Kamu masuk ke bidang ini dengan keunggulan yang tidak dimiliki mayoritas orang, dan keunggulan itu bukan “pintar matematika”. Keunggulannya adalah kamu sudah menguasai barangnya, dan yang perlu kamu pelajari cuma nama panggilannya di sini.

Yang kamu sudah tahu	Namanya di sini	Hubungannya
Energi potensial	Loss	Sistem bergerak menuju yang lebih rendah
Bola menggelinding ke lembah	Gradient descent	Ikuti kemiringan turun
Potensial harmonik $V = (1/2)kx^2$	Permukaan loss model linear	Bentuk yang sama, $k = 2A$
Osilasi karena kurang redaman	Divergensi karena learning rate kebesaran	Batas kestabilan yang sama
Hukum Hooke, gaya pemulih	Regularisasi L2	Pegas yang menarik bobot ke nol
Aturan rantai	Backpropagation	Aturan rantai dijalankan mundur
Hasil kali dalam bra-ket	Cosine similarity	Operasi identik, ruang berbeda
Keadaan ternormalkan	Embedding ternormalkan	Norma satu, arah yang bermakna

Yang kamu sudah tahu	Namanya di sini	Hubungannya
Distribusi Boltzmann	Softmax dengan temperature	Bentuk yang sama, T berperan sama
Fungsi partisi Z	Penyebut softmax	Penormal yang sama
Entropi $S = -k \sum p \log p$	Cross-entropy	Shannon meminjamnya dari sini
Deret Fourier	Positional encoding	Basis yang sama
Transformasi Fourier	Spektrogram	Fourier per potongan waktu
Teorema Nyquist	Sample rate audio	Batas yang sama
Aliasing	Artefak sampling	Gejala yang sama
Konvolusi, fungsi sebar titik	Lapisan konvolusi	Operasi sama, niat berbeda
Simulated annealing	Jadwal learning rate	Diambil dari mekanika statistik
Prinsip variasional	Optimasi	Cari titik stasioner fungsional
Lingkar kendali umpan balik	Agent loop	Diagram blok yang sama

Tabel ini bukan hiasan. Setiap kali kamu bertemu konsep baru dan merasa buntu, cari padanannya. Hampir selalu ada, karena orang yang membangun bidang ini banyak yang datang dari fisika dan mereka meminjam apa yang mereka kenal.

Bagian 10. Tes jujur, apakah kamu benar-benar paham

Ini bagian yang paling gampang dilewati dan paling penting.

Feynman punya cerita tentang ayahnya. Waktu kecil, teman-temannya menantang dia menyebut nama seekor burung. Dia tidak tahu. Mereka mengejek, katanya ayahnya tidak mengajarnya apa-apa.

Padahal ayahnya sudah mengajarnya begini: kamu bisa tahu nama burung itu dalam semua bahasa di dunia, dan setelah selesai kamu tetap tidak tahu apa-apa tentang burungnya. Lalu ayahnya menunjuk burung itu dan bertanya kenapa ia mematuhi bulunya, dan mereka menghabiskan sore itu mencari tahu.

Machine learning penuh dengan nama burung. Backpropagation. Attention. Embedding. Regularization. Kamu bisa hafal semuanya dan tetap tidak bisa membangun apa pun.

Jadi pakai daftar ini setiap kali kamu selesai satu bagian. Ini bukan kuis untuk dinilai orang lain. Ini alat supaya kamu tidak membohongi diri sendiri, dan diri sendiri adalah orang yang paling gampang kamu bohongi.

Uji satu, tanpa istilah. Jelaskan konsepnya ke orang yang tidak pernah kuliah, tanpa satu pun kata teknis. Kalau kamu terpaksa memakai istilah, kamu belum punya gambarannya, kamu baru punya labelnya.

Uji dua, dari sudut lain. Kamu bisa menjelaskan gradient descent. Sekarang jawab: kenapa ada tanda minus? Kenapa learning rate tidak dimasukkan saja ke dalam gradien? Kalau per-

tanyaan yang bentuknya sedikit berbeda langsung membuatmu buntu, yang kamu hafal adalah urutan penjelasan, bukan isinya.

Uji tiga, ramalkan dulu. Sebelum menjalankan kode, tulis apa yang kamu perkirakan akan keluar. Angkanya, bentuk grafiknya, arah perubahannya. Lalu jalankan. Setiap kali ramalanmu meleset, kamu baru saja menemukan lubang yang tidak kamu sadari ada. Ini alat paling tajam di daftar ini, dan paling jarang dipakai orang karena tidak nyaman.

Uji empat, rusakkan sengaja. Balik tanda gradien. Naikkan learning rate seratus kali. Buang fungsi aktivasi. Ramalkan dulu apa yang akan terjadi, lalu jalankan dan lihat apakah kamu benar. Kalau kamu bisa meramalkan cara sesuatu rusak, kamu paham cara kerjanya.

Uji lima, deteksi pemujaan kargo. Kamu memakai Adam dan bukan SGD. Kenapa? Kalau jawabanmu “karena semua orang pakai Adam” atau “karena di tutorial begitu”, itu bukan alasan, itu peniruan bentuk. Tidak apa-apa memakai sesuatu tanpa tahu alasannya, asal kamu jujur mencatat bahwa kamu belum tahu, dan tidak berpura-pura di depan diri sendiri bahwa kamu sudah memutuskan.

Feynman punya nama untuk gejala kelima ini. Setelah Perang Dunia II, penduduk beberapa pulau di Pasifik pernah melihat pesawat militer datang membawa barang. Setelah tentaranya pergi, mereka membangun landasan dari tanah, menara pengawas dari bambu, dan headphone dari batok kelapa, lalu ada yang berdiri di sisi landasan mengibaskan bendera. Semuanya persis seperti yang dulu mereka lihat.

Pesawatnya tidak datang.

Bentuknya sempurna. Isinya tidak ada. Dan kamu bisa melakukan hal yang sama pada kodemu tanpa menyadarinya: menumpuk lapisan karena arsitektur terkenal punya banyak lapisan, memasang batch normalization karena semua orang memasangnya, menyetel learning rate ke $3e-4$ karena angka itu sering muncul di mana-mana.

Obatnya cuma satu, dan tidak enak. Copot satu bagian, jalankan lagi, lihat apakah hasilnya memburuk. Kalau tidak, bagian itu adalah menara bambu.

Bagian 11. Peta satu halaman

Kalau kamu tersesat di tengah jalan, kembali ke halaman ini.

Semuanya adalah kelereng yang menggelinding di mangkuk. Mangkuknya adalah loss, ditentukan oleh datamu dan bentuk modelmu. Kelerengnya adalah parameter. Menggelinding adalah gradient descent.

Yang berubah antar modul cuma bentuk mangkuknya.

Modul 0 memberimu mangkuk dua dimensi yang bisa kamu plot dan lihat dengan mata sendiri.

Modul 1 menumpuk lapisan dan menekuknya, jadi mangkuknya bergelombang, dan kamu menulis mesin yang merasakan kemiringannya di ruang berdimensi ribuan.

Modul 2 memindahkan kata jadi tempat, sehingga kemiripan bisa diukur sebagai jarak, dan mangkuknya sekarang mengukur kejutan dengan cross-entropy.

Modul 3 mengubah suara jadi gambar lewat Fourier, lalu memakai stempel yang digeser untuk menemukan pola di gambar itu.

Modul 4 melatih model untuk membuat mangkuk yang menarik wajah yang sama saling mendekat dan mendorong yang berbeda saling menjauh.

Modul 5 memberi tiap kata kemampuan melihat kata lain dan mengambil yang ia butuhkan, dan mangkuknya jadi pegunungan berdimensi jutaan.

Modul 6 berhenti melatih dan mulai merakit, dan pekerjaannya berubah dari matematika jadi rekayasa.

Yang tidak pernah berubah: ukur seberapa salah, cari arah menurun, melangkah, ulangi.

Kalau kamu paham betul satu kalimat itu, sisanya cuma variasi.

Catatan penutup

Dokumen ini akan bikin kamu merasa sudah paham. Saya sebut ini di awal dan saya ulangi di akhir karena inilah jebakannya.

Kamu belum paham. Kamu baru punya peta.

Pemahaman datang saat gradient check kamu meleset di angka desimal ketiga dan kamu menghabiskan dua jam mencari tanda yang terbalik. Saat loss-mu berubah jadi nan di iterasi ketujuh dan kamu harus mencari tahu kenapa. Saat wake word buatanmu terpicu setiap kali kipas laptop menyala dan kamu harus memutuskan itu masalah ambang atau masalah data.

Jam-jam itu tidak bisa dipercepat dan tidak bisa diwakilkan. Dokumen ini cuma memastikan bahwa saat kamu ada di dalamnya, kamu tahu sedang mencari apa.

Satu hal terakhir. Kalau ada bagian di dokumen ini yang kamu baca dan rasanya “oh iya jelas”, tapi begitu diminta menjelaskan kamu buntu di kalimat kedua, jangan lewati. Itu bukan bagian yang mudah. Itu bagian yang belum kamu kunyah, menyamar jadi bagian yang mudah.

Itu saja isinya.

Dokumen terkait: [Silabus.md](#) untuk urutan dan tolok ukur, [Roadmap.md](#) untuk rencana besar, [Bulan-O-Harian.md](#) untuk rencana harian, [log.md](#) untuk catatan kerja.